

What is the Virtual DOM?

👉 The Virtual DOM (VDOM) is a lightweight copy of the actual DOM (Document Object Model) kept in memory by React.

👉 When your component's state or props change:

- 1 React updates the Virtual DOM first (in memory).
- 2 It compares (diffs) the new VDOM with the previous one.
- 3 React calculates the minimum number of changes needed.
- 4 React updates only those parts of the actual DOM (Real DOM).

★ Why does React use Virtual DOM?

- ✓ The **Real DOM** is slow to update because updating the browser UI is expensive.
- ✓ The **Virtual DOM** allows React to:

- Minimize direct manipulation of the real DOM
- Batch and optimize changes
- Make UI updates faster and more efficient

State/props change

↓

React creates new Virtual DOM

↓

React compares (diffs) old Virtual DOM and new Virtual DOM

↓

Finds what really changed

↓

Updates the real DOM efficiently (only the necessary parts)



Benefits of Virtual DOM

- ✓ Faster performance than directly manipulating the real DOM
- ✓ Reduces unnecessary re-renders or reflows
- ✓ Makes complex UIs more efficient
- ✓ Allows features like time-travel debugging (via state snapshots)

What is a React component?

- 👉 A **component** is a **reusable, self-contained piece of UI** in React.
- 👉 Components let you **split the UI into independent, reusable pieces** that manage their own structure, logic, and styling.
- ✓ Example: Button, Navbar, Card, FormField — all can be components!

Types of React components

There are **two main types**:



1 Function components (modern and preferred)

- ✓ A component that is just a JavaScript function returning JSX.

```
function Welcome(props) {  
  return <h1>Hello, {props.name}!</h1>;  
}
```

2 Class components (older style)

- ✓ A component that is a class extending `React.Component`.

```
class Welcome extends React.Component {  
  render() {  
    return <h1>Hello, {this.props.name}!</h1>;  
  }  
}
```

💡 Example: Combining components (Compose)

www.php-training.in / www.phpcourse.in

Mulund : 303,304,309 Shree Samarth Plaza , RRT Road, Opp. Mulund Station . Mulund West . Mumbai 400080

Vile Parle : Datapoint computers: vijaya Apt , Ground Floor ,Bajaj Road , Vile parle West, Vile Parle (West) . MUMBAI -56



```
function App() {
  return (
    <div>
      <Header />
      <MainContent />
      <Footer />
    </div>
  );
}

function Header() {
  return <h1>My App</h1>;
}

function MainContent() {
  return <p>Welcome to my app!</p>;
}

function Footer() {
  return <p>(c) 2025 My Company</p>;
}
```

⚡ Key features of React components

Reusable Use the same component multiple times

www.php-training.in / www.phpcourse.in

Mulund : 303,304,309 Shree Samarth Plaza , RRT Road, Opp. Mulund Station . Mulund West . Mumbai 400080

Vile Parle : Datapoint computers: vijaya Apt , Ground Floor ,Bajaj Road , Vile parle West, Vile Parle (West) . MUMBAI -56



```
Composable  Nest components inside each other
Maintainable  Organize code into logical parts
Isolated     Each component manages its own logic and appearance
```

✨ Summary

Function
component

```
function MyComp() {}
```

Hooks support, cleaner
code

Class component

```
class MyComp extends  
  React.Component {}
```

Lifecycle methods, older
style

What are props in React?

- Props (short for properties) are read-only data passed from a parent component to a child component.
- Props allow components to be dynamic and reusable by passing different data.

Pass props

www.php-training.in / www.phpcourse.in

Mulund : 303,304,309 Shree Samarth Plaza , RRT Road, Opp. Mulund Station . Mulund West . Mumbai 400080

Vile Parle : Datapoint computers: vijaya Apt , Ground Floor ,Bajaj Road , Vile parle West, Vile Parle (West) . MUMBAI -56



```
<Welcome name="John" age={25} />
```

Receive props in a Function component

```
function Welcome(props) {  
  return (  
    <h1>Hello, {props.name}! You are {props.age}  
years old.</h1>  
  );  
}
```

Receive props in a Class component

```
class Welcome extends React.Component {  
  render() {
```

```
return (  
    <h1>Hello, {this.props.name}! You are  
{this.props.age} years old.</h1>  
    );  
}
```

What is a state variable in React?

- A state variable holds dynamic data that can change over time and affect what gets rendered in the UI.
- When a state variable updates, React re-renders the component to reflect the new data.
- State is local to the component and managed internally.

State variable in CLASS components

```
class MyComponent extends React.Component {  
  constructor(props) {  
    super(props);  
    this.state = {  
      count: 0,      // state variable  
      name: "React" // another state variable  
    }  
  }  
}
```

```
    };  
  }  
  
  render() {  
    return (  
      <div>  
        <p>Count: {this.state.count}</p>  
        <p>Name: {this.state.name}</p>  
      </div>  
    );  
  }  
}
```

Update state variable using `setState()`:

```
this.setState({ count: this.state.count + 1 });
```

What is `setState`?

- `setState` is a method used in **class components** to update the component's **state** and tell React to re-render the component with the new state.
- When you call `setState`, React schedules an update to the component's state object and tells React that this component and its children need to be re-rendered.

```
class Counter extends React.Component {  
  constructor(props) {  
    super(props);  
    this.state = { count: 0 };  
  }  
}
```

```
}

increment = () => {
  this.setState({ count: this.state.count + 1 });
};

render() {
  return (
    <div>
      <p>Count: { this.state.count }</p>
      <button onClick={this.increment}>Increment</button>
    </div>
  );
}
}
```

React Class Component Lifecycle

In a React class component, the lifecycle consists of **three main phases**:

- ❶ **Mounting** (component is being created and inserted into the DOM)
- ❷ **Updating** (component is being re-rendered due to changes in props or state)
- ❸ **Unmounting** (component is being removed from the DOM)

```
class DemoComponent extends React.Component {
```



```
constructor(props) {
  super(props);
  this.state = { count: 0 };
  console.log('constructor');
}

componentDidMount() {
  console.log('componentDidMount');
}

componentDidUpdate(prevProps, prevState, snapshot) {
  console.log('componentDidUpdate');
}

componentWillUnmount() {
  console.log('componentWillUnmount');
}

componentDidCatch() {
  console.log('componentDidCatch');
}

render() {
  console.log('render');
  return (
    <div>
    </div>
  );
}
```



What is componentDidMount?

- `componentDidMount()` is a lifecycle method in **React class components**.
- It is called **once** — immediately after a component is **mounted** (inserted into the DOM).

What is componentDidUpdate?

- `componentDidUpdate(prevProps, prevState, snapshot)` is a **lifecycle method** in class components.
- It is called **immediately after a component updates** (i.e., after `render()` when props or state changes).

What is componentWillUnmount?

- `componentWillUnmount()` is a **lifecycle method** in class components.
- It is called **right before** a component is removed (unmounted) from the DOM.

What is componentDidCatch?

- `componentDidCatch(error, info)` is a **React lifecycle method** that lets class components catch JavaScript errors **anywhere in their child component tree**.
- It helps you build **error boundaries** — components that catch errors and display a fallback UI rather than crashing the whole app.

What is event handling in React?

👉 Event handling in React lets you respond to user interactions like:

- ✓ Clicks
- ✓ Form submissions
- ✓ Mouse movements
- ✓ Keyboard inputs

⚡ The syntax is **similar to DOM event handling**, but with some React-specific differences!

```
function Button() {  
  const handleClick = () => {  
    alert('Button clicked!');  
  };  
  
  return <button onClick={handleClick}>Click Me</button>;  
}
```

⚡ Common events in React

```
onClick Handle clicks  
onChange Handle input field changes
```



```
onSubmit  Handle form submission
onMouseEnter / onMouseLeave Mouse hover in/out
onKeyDown / onKeyUp Keyboard key presses
```

Passing arguments to event handlers

```
function Item({ name }) {
  const handleClick = (itemName) => {
    alert(`You clicked on ${itemName}`);
  };

  return (
    <button onClick={() => handleClick(name)}>Click {name}</button>
  );
}
```

What is **map()** in React?

- 👉 **map()** is a JavaScript array method that lets you loop over an array and transform each item.
- 👉 In React, you use **map()** to generate JSX elements dynamically for each item in a list.

```
function NameList() {
  const names = ["Alice", "Bob", "Charlie"];

  return (
    <ul>
      {names.map((name, index) => (
        <li key={index}>{name}</li>
      ))}
    </ul>
  );
}
```

```
    )})  
  </ul>  
  );  
}
```

Example: dynamic rendering with map

```
function ProductList() {  
  const products = [  
    { id: 101, name: "Laptop", price: 50000 },  
    { id: 102, name: "Phone", price: 20000 },  
    { id: 103, name: "Tablet", price: 30000 }  
  ];  
  
  return (  
    <div>  
      {products.map(product => (  
        <div key={product.id} style={{ border: '1px solid #ccc', margin: '5px',  
padding: '5px' }}>  
          <h4>{product.name}</h4>  
          <p>Price: ₹{product.price}</p>  
        </div>  
      )})  
    </div>  
  );  
}
```

☀ What is Composition in React?

👉 Composition means:

- ✓ Building components by combining simpler components.
- ✓ Nesting components inside each other.
- ✓ Passing JSX, functions, or data as props or children.

```
function Card({ title, children }) {  
  return (  
    <div className="card">  
      <h2>{title}</h2>  
      <div className="content">{children}</div>  
    </div>  
  );  
}  
  
function App() {  
  return (  
    <Card title="Welcome">  
      <p>This is a card with content inside.</p>  
    </Card>  
  );  
}
```

☀ What is Inheritance in React?

👉 **Inheritance** would mean creating components by extending another component's class or logic.

```
class BaseComponent extends React.Component {
  render() {
    return <div>Base content</div>;
  }
}

class ExtendedComponent extends BaseComponent {
  render() {
    return (
      <div>
        {super.render()}
        <p>Extended content</p>
      </div>
    );
  }
}
```

What is an Error Boundary?

- An **Error Boundary** is a React component that **catches JavaScript errors** in its child component tree (during render, lifecycle methods, and constructors) and displays a fallback UI **instead of crashing the entire app**.

- It prevents the app from breaking entirely due to errors in parts of the UI.

When to use Error Boundaries

- ✓ Around components that might fail (e.g. complex widgets, external modules, third-party components)
- ✓ Around large sections of your app (so one part failing doesn't break the whole app)

```
class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false };
  }

  static getDerivedStateFromError(error) {
    // Update state so the next render shows fallback UI
    return { hasError: true };
  }

  componentDidCatch(error, info) {
    // You can also log the error to an error reporting service
    console.log("Caught an error:", error);
    console.log("Error info:", info.componentStack);
  }

  render() {
    if (this.state.hasError) {

```



```
// Render fallback UI
return <h1>Something went wrong.</h1>;
}

return this.props.children;
}
}

<ErrorBoundary>
  <MyComponent />
</ErrorBoundary>
```

Hooks in React?

What is `useState`?

`useState` is a **hook** introduced in **React 16.8** that allows you to add **state variables** to **function components**.

It replaces the need for `this.state` and `this.setState` used in class components.

Basic syntax:

```
const [stateVariable, setStateFunction] = useState(initialValue);
```

```
import React, { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0); // count is state variable, initial
```

```
value 0

return (
  <div>
    <p>Count: {count}</p>
    <button onClick={() => setCount(count + 1)}>
      Increment
    </button>
  </div>
);
}
```

Rules of **useState**

- ⚠ Hooks must be **called at the top level** of your component (not inside loops, conditions, or nested functions).
- ⚠ Hooks must be called **inside a React function component or a custom hook**.

What is **useEffect**?

- **useEffect** lets you perform **side effects** in your **function components**.
- Side effects include things like:
 - ✓ Data fetching (API calls)
 - ✓ Subscribing to events (e.g. WebSocket, DOM event)
 - ✓ Manipulating the DOM



✓ Setting up timers

👉 **useEffect** replaces lifecycle methods like:

- `componentDidMount`
- `componentDidUpdate`
- `componentWillUnmount`

```
useEffect(() => {  
  // Your side effect code  
  
}, [dependencies]);
```

useEffect Hook With API Call:

```
import React, { useEffect, useState } from 'react';  
  
function UserList() {  
  const [product, setProduct] = useState([]);  
  
  useEffect(() => {  
    // Runs on mount (empty dependency array)  
    fetch('https://fakestoreapis.com/products')  
      .then(res => res.json())  
      .then(data => setProduct(data));  
  }, []);  
  
  return (  

```

```
<ul>
  {product.map(product => <li>{product.title}</li>)}
</ul>
);
}
```

What is **useRef**?

- **useRef** is a hook that gives you a mutable ref object.
- That object:
 - ✓ persists between renders (doesn't reset)
 - ✓ doesn't cause a re-render when changed
 - ✓ can store a DOM element reference or any mutable value

👉 It's often used to:

- Access DOM nodes directly
- Keep a value that survives re-renders without triggering them
- Store instance variables in function components

Basic syntax

```
const myRef = useRef(initialValue);
```

Example 1: Accessing a DOM element

```
import React, { useRef, useEffect } from 'react';

function InputFocus() {
  const inputRef = useRef();

  useEffect(() => {
    // Focus the input element on mount
    inputRef.current.focus();
  }, []);

  return <input ref={inputRef} type="text" placeholder="Focus on load" />;
}
```

What is contextAPI?

Context API allows you to share data (state, functions, etc.) globally across your component tree — without passing props manually at every level (no more “prop drilling”).

🌟 When to use Context

- 👉 When you have **data that many components at different nesting levels need access to**.
- 👉 When you want to avoid passing props down multiple levels unnecessarily.

Create a Context

www.php-training.in / www.phpcourse.in

Mulund : 303,304,309 Shree Samarth Plaza , RRT Road, Opp. Mulund Station . Mulund West . Mumbai 400080

Vile Parle : Datapoint computers: vijaya Apt , Ground Floor ,Bajaj Road , Vile parle West, Vile Parle (West) . MUMBAI -56

```
const MyContext = React.createContext(defaultValue);
```

Provide the context at a high level

```
<MyContext.Provider value={someValue}>
  <App />
</MyContext.Provider>
```

Consume the context in any child component

```
const value = useContext(MyContext);

<MyContext.Consumer>
  {value => <div>{value}</div>}
</MyContext.Consumer>
```

What is prop drilling?

- **Prop drilling** happens when you pass props through **many intermediate components** just to get them to a deeply nested child.
- Intermediate components don't use the prop themselves — they simply pass it along.

👉 This can make your code:

- ✓ Harder to read
- ✓ Harder to maintain
- ✓ More prone to bugs

Example of prop drilling

```
function App() {  
  const user = { name: "Alice" };  
  return <Parent user={user} />;  
}  
  
function Parent({ user }) {  
  return <Child user={user} />;  
}  
  
function Child({ user }) {  
  return <GrandChild user={user} />;  
}  
  
function GrandChild({ user }) {  
  return <p>Hello, {user.name}</p>;  
}
```

What is state lifting in React?

 State lifting means moving state to the closest common ancestor of two or more components that need to share or coordinate that state.

www.php-training.in / www.phpcourse.in

Mulund : 303,304,309 Shree Samarth Plaza , RRT Road, Opp. Mulund Station . Mulund West . Mumbai 400080

Vile Parle : Datapoint computers: vijaya Apt , Ground Floor ,Bajaj Road , Vile parle West, Vile Parle (West) . MUMBAI -56

👉 The lifted state is passed down to children via props, and child components can inform the parent to update the state (usually through callback functions).

🌟 Why lift state up?

- ✓ To avoid duplicate or inconsistent state in sibling components
- ✓ To coordinate behavior between components
- ✓ To follow React's one-way data flow model

🚀 Example: Lifting state up

```
function App() {  
  const [count, setCount] = React.useState(0);  
  
  return (  
    <>  
      <Sibling1 count={count} setCount={setCount} />  
      <Sibling2 count={count} />  
    </>  
  );  
}
```

```
function Sibling1({ count, setCount }) {  
  return <button onClick={() => setCount(count + 1)}>Count: {count}</button>;  
}  
  
function Sibling2({ count }) {  
  return <p>Sibling2 sees: {count}</p>;  
}
```



How to lift state

- ✓ Identify the closest common parent of components that need the shared data.
- ✓ Move the state to that parent.
- ✓ Pass the state and **setState** (or handler functions) down as props.



When to lift state

- 👉 When multiple components depend on or modify the same data
- 👉 When siblings need to coordinate
- 👉 When state duplication causes inconsistencies



What is Redux?

- Redux is a state management library for JavaScript apps (commonly used with React).
- It helps you manage global state in a predictable, centralized way.

- Useful when:

- ✓ Many components need to access or modify the same data
- ✓ Your app's state becomes complex
- ✓ You want time-travel debugging or predictable updates

👉 Core principles of Redux:

- 1 Single source of truth → State is stored in one place: the Redux store.
- 2 State is read-only → You can't change state directly.
- 3 Changes via pure functions (reducers) → You dispatch actions, reducers compute new state.

🌟 Key concepts

```
Create reducer + actions : createSlice
Setup store : configureStore
Dispatch actions : useDispatch + slice.actions
Get state : useSelector
```

```
// App.js
import React from 'react';
import { Provider, useSelector, useDispatch } from 'react-redux';
import { store } from './app/store';
import { increment, decrement, incrementByAmount } from
'./features/counterSlice';

function Counter() {
  const count = useSelector((state) => state.counter);
  const dispatch = useDispatch();

  return (
```

```
<div>
  <h2>Count: {count}</h2>
  <button onClick={() => dispatch(increment())}>+</button>
  <button onClick={() => dispatch(decrement())}>-</button>
  <button onClick={() => dispatch(incrementByAmount(5))}>+5</button>
</div>
);
}

function App() {
  return (
    <Provider store={store}>
      <Counter />
    </Provider>
  );
}

export default App;
```

Why is a Redux store required?

The **Redux store** is the **single source of truth** that holds and manages the global state of your application in a **predictable** way.

Here's **why it's needed**:

1 Centralized state management



- ✓ Instead of scattered state in many components, **all state lives in one place (the store)**.
 - ✓ This makes it easier to **share data between components** (no prop drilling).
-

2 Predictable state updates

- ✓ State can only be updated by **dispatching actions**.
 - ✓ Updates go through **reducers**, which are pure functions → always predictable.
 - ✓ Helps prevent bugs from accidental state changes.
-

3 Consistent one-way data flow

- ✓ The store ensures that state flows in one direction: